

CodeAlive

Real-Time Collaboration Feature

Feature Design & Architecture Document

Version 1.0 · April 2026 · Internal Planning

Document Purpose

This document captures the complete design, architecture, data model, WebSocket protocol, edge case analysis, and phased build plan for adding real-time collaboration rooms to CodeAlive. It is intended as a planning reference before any code is written.

1. Feature Overview

CodeAlive currently operates as a stateless, anonymous code-sharing tool. Users paste code, receive a shareable link, and anyone with the link can view that snippet. No login is required for this core flow, and it will remain unchanged.

The collaboration feature adds a separate, opt-in layer on top: authenticated users can create rooms, share a room URL, and have other authenticated users join. Everyone in a room sees live code changes and line highlights in real time.

1.1 What Changes vs. What Stays the Same

Area	Behaviour
Anonymous snippet sharing	Unchanged. No login required. No impact whatsoever.
Syntax highlighting (Prism)	Unchanged. Runs locally on every client independently.
Language detection	Unchanged. Runs locally on every client with its own debounce.
Multi-language detection (upcoming)	Unchanged. Purely a local editor feature.
Image strip panel (upcoming)	Unchanged. Local UI feature.
Room creation	New. Requires login.
Room joining / approval	New. Requires login. Host approves each joiner.
Real-time code sync	New. WebSocket broadcasts raw code string only.
Real-time line highlight sync	New. WebSocket broadcasts selection range only.

1.2 Design Principles

- Anonymous use is never broken or degraded by this feature.
- WebSockets carry the minimum possible data — raw code string and line selection range only.
- All rendering (syntax highlighting, language detection, formatting) is local on each client.
- For v1, only the host can type. All other participants are read-only viewers.
- The host controls who enters the room (Zoom-style approval flow).
- Room state is persisted in PostgreSQL. WebSocket connections are ephemeral.

2. Authentication & Access Model

CodeAlive does not currently have user accounts. This feature introduces a lightweight auth system scoped only to the collaboration feature. The pattern mirrors the DataInsights JWT implementation.

2.1 Auth Stack

- JWT tokens containing `userId`, `email`, and `displayName`.
- Stateless validation — no sessions table needed for v1.
- JWT passed as a Bearer token on HTTP requests, and as a query parameter on WebSocket connections (e.g. `/ws/room/{room_id}?token=...`).
- Password hashing via `bcrypt`.

2.2 Who Needs Login

Action	Auth Required?
Paste and share a code snippet	No — anonymous as today
View a shared snippet link	No — anonymous as today
Create a room	Yes
Request to join a room	Yes
View room as approved participant	Yes
Click a room URL while logged out	Redirected to login, then back to the room

3. Database Schema

All room and user data lives in PostgreSQL, consistent with the existing DataInsights pattern. Redis is used only for ephemeral WebSocket connection tracking and pub/sub broadcasting, not for room data persistence.

3.1 PostgreSQL Tables

users

```
CREATE TABLE users (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email       TEXT UNIQUE NOT NULL,  
  password_hash TEXT NOT NULL,  
  display_name TEXT NOT NULL,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);
```

rooms

```
CREATE TABLE rooms (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  host_id     UUID REFERENCES users(id) ON DELETE CASCADE,  
  title       TEXT,  
  code_snapshot TEXT DEFAULT '',  
  language    TEXT DEFAULT 'plaintext',  
  is_active   BOOLEAN DEFAULT TRUE,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  last_active_at TIMESTAMPTZ DEFAULT NOW()  
);
```

room_participants

```
CREATE TABLE room_participants (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  room_id     UUID REFERENCES rooms(id) ON DELETE CASCADE,  
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,  
  joined_at   TIMESTAMPTZ DEFAULT NOW(),  
  UNIQUE(room_id, user_id)  
);
```

room_join_requests

```
CREATE TABLE room_join_requests (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  room_id     UUID REFERENCES rooms(id) ON DELETE CASCADE,  
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,  
  status      TEXT DEFAULT 'pending', -- pending | approved | rejected  
  requested_at TIMESTAMPTZ DEFAULT NOW(),  
  resolved_at TIMESTAMPTZ,  
  UNIQUE(room_id, user_id)
```

```
);
```

3.2 Redis Usage

Redis Key	Purpose
ws:room:{room_id}	In-memory set of active socket connection IDs for a room
channel:room:{room_id}	Pub/sub channel — broadcasts messages to all room subscribers

Note: Redis holds zero source-of-truth data. If Redis is flushed or the server restarts, all WebSocket connections drop, clients reconnect, and PostgreSQL provides the recovery snapshot.

4. WebSocket Protocol

The WebSocket connection is established at `/ws/room/{room_id}?token={jwt}`. Once open it remains persistent for the duration of the session. Only two message types travel over the wire.

4.1 Exactly Two Message Types

Design Decision

Language detection, syntax highlighting, Prism rendering, formatting, and multi-language detection are all local operations. They run independently on every client. They are never sent over the WebSocket. This keeps the protocol minimal and ensures broadcast payloads are tiny.

code_change — sent by host on editor input (debounced 300ms)

```
// Client (host) → Server → All participants
{
  "type": "code_change",
  "code": "def hello():\n    pass"
}
```

selection — sent by any participant on text selection (debounced 50ms)

```
// Client → Server → All other participants
{
  "type": "selection",
  "start_line": 4,
  "end_line": 7
}
```

4.2 Server-to-Client Control Messages

The server sends additional control messages to manage the room lifecycle. These are one-directional — server to client only.

Message Type	Payload & When Sent
join_request	{ user_id, display_name } — Sent to host when a new user requests to join
join_approved	{ code_snapshot, language } — Sent to the approved user with current state
join_rejected	{ } — Sent to the rejected user
participant_joined	{ display_name, color } — Broadcast to all when someone is approved
participant_left	{ display_name } — Broadcast to all when a participant disconnects
selection_cleared	{ user_id } — Broadcast when a participant disconnects, to clear their highlight

host_disconnected	{ } — Broadcast to all when host's socket closes
host_reconnected	{ } — Broadcast to all when host reconnects
room_closed	{ } — Broadcast to all when host explicitly closes the room
error	{ code, message } — e.g. room_full, room_inactive, auth_failed

4.3 Debounce Strategy

Two separate debounce timers run independently on the frontend. They are not connected to the language detection debounce, which is a third independent timer.

Event	Debounce Delay	Rationale
code_change (host typing)	300ms	Feels real-time. Prevents per-keystroke floods. At 5 chars/sec, max ~3 messages/sec.
selection (line highlight)	50ms	Must feel instant to other viewers. Lightweight payload justifies low delay.
Language detection (local)	1200ms	Heavy API call. Independent — not a WebSocket message.

5. Room Join Flow

5.1 Creating a Room

1. Authenticated user clicks 'Create Room' in CodeAlive.
2. Frontend calls POST `/rooms/create` with JWT.
3. Server creates room in PostgreSQL, returns `room_id`.
4. Frontend redirects to `/room/{room_id}` — this is the shareable URL.
5. Host's browser opens WebSocket to `/ws/room/{room_id}?token={jwt}`.
6. Server validates JWT, confirms host identity, admits immediately.

5.2 Joining a Room (Participant Flow)

7. Participant opens the shared room URL.
8. If not logged in: redirected to `/login?redirect=/room/{room_id}`, then back after login.
9. Browser opens WebSocket to `/ws/room/{room_id}?token={jwt}`.
10. Server checks `room_participants` — if already approved (reconnection), admits with snapshot immediately.
11. If not previously approved: server inserts into `room_join_requests` with `status='pending'`, sends `join_request` message to host.
12. Participant sees 'Waiting for host to approve...' screen.
13. Host sees approval popup with participant's display name.
14. Host approves: server updates request to `status='approved'`, inserts into `room_participants`, sends `join_approved` with current code snapshot to participant, broadcasts `participant_joined` to all.
15. Host rejects: server updates request to `status='rejected'`, sends `join_rejected` to participant.

5.3 Reconnection

If a participant's WebSocket drops (flaky internet, tab refresh), they reconnect silently without needing host approval again. The server checks `room_participants`, finds the existing approved record, and sends the current code snapshot immediately. From the host's perspective, nothing happens.

6. Edge Cases & Handling

6.1 Connection & Sync

Late Joiner

A participant joins a room where substantial code already exists. On `join_approved`, the server reads the latest `code_snapshot` from PostgreSQL (kept updated periodically) and includes it in the `join_approved` payload. The participant's editor initialises with the current state, not an empty buffer.

Host Disconnects Mid-Session

Host's WebSocket closes unexpectedly. Server broadcasts `host_disconnected` to all participants. Participant editors become read-only. A status banner shows 'Host disconnected — waiting for reconnect...'. Room is not closed. When host reconnects, server broadcasts `host_reconnected`, editor re-enables for host.

Participant Disconnects

Server detects WebSocket close event. Broadcasts `participant_left` and `selection_cleared` for that user to all remaining participants. Their highlight is removed from everyone's view. Their record in `room_participants` remains — they can rejoin silently.

Server Restart

All in-memory WebSocket connections are lost. Every client receives a connection close event and the frontend auto-attempts reconnection with exponential backoff (e.g. 1s, 2s, 4s, 8s). A 'Reconnecting...' banner is shown. On successful reconnect, each client receives the latest snapshot from PostgreSQL. No data is lost.

6.2 Room Lifecycle

Pending Request Timeout

A user sends a join request but the host never responds (busy, tabbed away, AFK). After 2 minutes, the server auto-rejects: updates status to 'rejected', sends `join_rejected` to the waiting participant with the message 'Host did not respond'. The participant is not left waiting indefinitely.

Room Participant Cap

If a room already has 50 approved participants and a new user tries to join, the server sends an error message with `code='room_full'` immediately. The join request is never forwarded to the host.

Room Inactivity Expiry

If `last_active_at` is more than 24 hours ago, the room is considered expired. New connection attempts receive an error with `code='room_inactive'`. The room record is not deleted — historical data is preserved.

Host Closes Room

Host clicks 'End Room'. Server sets `is_active=false` in PostgreSQL, broadcasts `room_closed` to all connected participants, and closes all WebSocket connections gracefully. Participants see a 'This room has ended' screen.

6.3 Highlight / Selection Edge Cases

Highlight on Lines That No Longer Exist

Host deletes lines 8–20 while participant B has lines 10–15 highlighted. The highlight now references non-existent line numbers. Resolution: on every `code_change` broadcast received by a client, all remote highlights are cleared. This is simpler and safer than remapping line numbers.

Multiple Participants Highlighting Simultaneously

Each participant is assigned a colour from a fixed palette on join (e.g. orange, green, purple, teal). The colour is included in the `participant_joined` broadcast. Each client stores a local map of `{user_id: color}` and renders each user's selection in their assigned colour.

Disconnected User's Highlight Lingers

If a participant disconnects abruptly without sending a `deselect`, their highlight remains visible on other screens. The server handles this: on WebSocket close event, it broadcasts `selection_cleared` for that `user_id`, so all clients remove the ghost highlight.

6.4 Auth Edge Cases

JWT Expires Mid-Session

JWT is validated only at WebSocket connection time. Once a connection is open and the participant is approved, they remain connected for the session regardless of token expiry. Token refresh is only needed on new connections. This is safe because the WebSocket connection itself is the session.

Same User in Two Tabs

Two WebSocket connections from the same `user_id` are allowed. Both tabs receive all broadcasts. The user is counted once in the participants list. If one tab closes, the other remains connected. The user is only removed from participants when all their connections are gone.

Unauthenticated User Clicks Room URL

Frontend detects no JWT in localStorage (or expired JWT). Before redirecting to /login, the room URL is stored — either in localStorage as intended_redirect or as a query param: /login?redirect=/room/{room_id}. After successful login, the user is sent directly to the room.

7. Backend File Structure

New files to add to the FastAPI backend. Zero existing files are modified.

New File	Responsibility
auth_router.py	POST /auth/register, POST /auth/login — returns JWT
rooms_router.py	POST /rooms/create, GET /rooms/{room_id} — room HTTP endpoints
ws_router.py	WebSocket /ws/room/{room_id} — connection handler, approval loop, broadcast relay
room_service.py	Room CRUD, participant management, snapshot persistence
redis_pubsub.py	Pub/sub helpers — publish to channel, subscribe, broadcast to all room connections
auth_utils.py	JWT sign/verify, bcrypt helpers

7.1 WebSocket Handler Logic (ws_router.py)

16. Accept incoming WebSocket connection.
17. Extract and validate JWT from query param. Reject with close code 4001 if invalid.
18. Look up room in PostgreSQL. Reject with close code 4004 if inactive or not found.
19. Check room_participants — if user already approved, send current snapshot and begin relay loop.
20. Otherwise, insert into room_join_requests, send join_request to host's socket, enter pending loop.
21. Pending loop: wait for host approval message. Timeout after 120 seconds.
22. On approval: subscribe to Redis channel, enter broadcast relay loop.
23. Relay loop: receive message from client → validate type (code_change or selection only) → publish to Redis channel → all subscribers forward to their respective WebSocket clients.
24. On WebSocket close: remove from active connections, broadcast participant_left and selection_cleared, update code_snapshot in PostgreSQL if host.

8. Frontend Changes

The existing editor, detection.js, ui.js, editor.js, main.js, and codec.js are untouched. New frontend modules are added alongside them.

8.1 New Modules

New File	Responsibility
room.js	Room creation, join request, WebSocket lifecycle management, message dispatch
participants-ui.js	Participants sidebar — list of who's in the room, colour assignments
approval-ui.js	Host approval popup — 'X wants to join' with Accept/Reject buttons
highlight-overlay.js	Render per-user line highlights in their assigned colours over the editor

8.2 Editor Behaviour Changes

- Host editor: sends code_change via WebSocket on input, debounced 300ms. Editor remains fully editable.
- Participant editor: receives code_change broadcasts, updates raw content. Editor is set to contenteditable=false — read-only. Text selection (Ctrl+C, copy) still works.
- All editors: send selection (line highlight) via WebSocket on selectionchange event, debounced 50ms.
- All editors: continue running Prism, language detection, formatting locally on every keystroke as today.

9. Phased Build Plan

Phase	What Gets Built	Done When...
1 — Auth	users table, /auth/register, /auth/login, JWT middleware, login UI page	Can register, log in, receive JWT
2 — Rooms	rooms table, /rooms/create, /rooms/{id}, room URL routing, Create Room UI button	Can create room, get shareable URL
3 — WebSocket Core	ws_router.py, Redis pub/sub, join request flow, approval popup, join_approved with snapshot	Two browsers in same room, host types, participant sees changes
4 — Highlights	selection messages, highlight-overlay.js, participant colour assignment, selection_cleared on disconnect	Participants see each other's line highlights in colour
5 — Polish	Reconnection with backoff, host disconnect banner, room expiry, participant cap, request timeout, room_closed flow	All edge cases handled gracefully

10. Explicitly Out of Scope for V1

Design Boundary

The following capabilities are deliberately excluded from v1. They represent significantly more complex engineering and are documented here to prevent scope creep.

Feature	Why Deferred
Multi-user simultaneous editing	Requires Operational Transformation (OT) or CRDTs. Genuinely complex algorithms. Not needed when host-only editing is the model.
Conflict resolution	Not applicable when only one user (host) can type.
Voice or video	Out of scope entirely. Use Zoom/Meet alongside CodeAlive.
Chat / comments	Out of scope for v1. Can be added later as a separate WebSocket message type.
Cursor position sync (character-level)	Line-level highlight is sufficient for v1. Character cursor requires more complex overlay logic.
Refresh tokens / JWT rotation	Stateless JWTs are sufficient for v1. Add refresh tokens when session length becomes a user concern.
Room recording / replay	Interesting feature but out of scope.
Permissions beyond host/viewer	No co-host or editor roles in v1.

11. Summary

The collaboration feature is a well-scoped addition to CodeAlive. The core anonymous snippet flow is completely unaffected. The WebSocket protocol is intentionally minimal — two message types for raw code and line selection only. All rendering remains local on every client.

PostgreSQL owns all room and user data. Redis handles ephemeral connection state and pub/sub. FastAPI's native WebSocket support is sufficient — no external libraries like Socket.io are needed.

The host-only editing model eliminates conflict resolution complexity for v1 while still delivering the core value: multiple people watching the same code evolve in real time, with live line highlights showing what's being worked on.

Next Step

Build the two pending CodeAlive features (image strip panel, multi-language detection) first. Then begin Phase 1 of this feature: auth endpoints and the users table.